

Punto 5.10 — Logging y diagnóstico en Entity Framework Core

Práctica

Ejercicio: Integrar EF Core con ILogger y Serilog en el proyecto AceriaData. Configurar el logging de las operaciones de escritura y las consultas. Registrar los mensajes en la consola y en un archivo. Analizar los logs generados.

Contexto del proyecto: En el punto 5.9 se consolidó el patrón Repositorio y la unidad de trabajo en aplicaciones empresariales. En este punto se profundiza en el logging y el diagnóstico, que son esenciales para monitorizar el comportamiento de EF Core en producción. Esta técnica se usará en el punto 5.11 para el testing con EF Core.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Instalar los paquetes de Serilog

```
dotnet add package Serilog.Extensions.Logging
dotnet add package Serilog.Sinks.Console
dotnet add package Serilog.Sinks.File
```

dotnet add package Serilog.Extensions.Logging → añade la integración con ILoggerFactory. dotnet add package Serilog.Sinks.Console → añade el proveedor de consola. dotnet add package Serilog.Sinks.File → añade el proveedor de archivo.

Error común: si se olvida alguno de los paquetes, la configuración de Serilog no compila. Se deben instalar los tres.

Paso 3: Configurar Serilog en el método Main

Modificar el método Main de src/AceriaData.Console/Program.cs:

```
using Serilog;

public static void Main()
{
    Log.Logger = new LoggerConfiguration()
        .MinimumLevel.Information()
        .WriteTo.Console()
        .WriteTo.File("logs/aceria-.log", rollingInterval: RollingInterval.Day)
        .CreateLogger();

    try
    {
        Log.Information("Iniciando la aplicación AceriaData");

        var configuration = new ConfigurationBuilder()
            .SetBasePath(Directory.GetCurrentDirectory())
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: true)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");
```

```

var services = new ServiceCollection();

services.AddLogging(builder =>
{
    builder.AddSerilog(Log.Logger, dispose: true);
});

services.AddDbContext<AceriaDbContext>(options =>
    options
        .UseSqlServer(connectionString, sqlOptions =>
            {
                sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);
                sqlOptions.CommandTimeout(60);
            })
        .EnableSensitiveDataLogging()
        .EnableDetailedErrors());

services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();
services.AddScoped<LoggingDagnosticoUseCase>();

_provider = services.BuildServiceProvider();

using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<LoggingDagnosticoUseCase>();
    useCase.Ejecutar();
}
}
catch (Exception ex)
{
    Log.Fatal(ex, "La aplicación ha terminado debido a una excepción no controlada");
}
finally
{
    Log.CloseAndFlush();
}
}

```

Línea 1: using Serilog; → importa el espacio de nombres de Serilog.

Línea 3: public static void Main() → punto de entrada.

Línea 5: Log.Logger = new LoggerConfiguration() → crea la configuración de Serilog.

Línea 6: .MinimumLevel.Information() → establece el nivel mínimo.

Línea 7: .WriteTo.Console() → escribe en la consola.

Línea 8: .WriteTo.File("logs/aceria-.log", rollingInterval: RollingInterval.Day) → escribe en un archivo diario.

Línea 9: .CreateLogger(); → crea el logger.

Línea 11: try → inicio del bloque.

Línea 13: Log.Information("Iniciando la aplicación AceriaData"); → registra el inicio.

Línea 15: var configuration = new ConfigurationBuilder() → crea el constructor de configuración.

Línea 19: .Build(); → construye la configuración.

Línea 21: var connectionString = configuration.GetConnectionString("AceriaDB") → lee la cadena de conexión.

Línea 24: var services = new ServiceCollection(); → crea la colección de servicios.

Línea 26: services.AddLogging(builder => → registra el logging.

Línea 28: builder.AddSerilog(Log.Logger, dispose: true); → añade Serilog.

Línea 29: }); → cierra la configuración del logging.

Línea 31: `services.AddDbContext(options => →` registra el `DbContext`.

Línea 40: `.EnableDetailedErrors()); →` muestra información detallada en los errores.

Línea 42: `services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>()); →` registra la unidad de trabajo.

Línea 43: `services.AddScoped(); →` registra el caso de uso.

Línea 45: `_provider = services.BuildServiceProvider(); →` construye el proveedor.

Línea 47: `using (var scope = _provider.CreateScope()) →` crea un ámbito.

Línea 49: `var useCase = scope.ServiceProvider.GetRequiredService(); →` resuelve el caso de uso.

Línea 50: `useCase.Ejecutar(); →` ejecuta el caso de uso.

Línea 53: `catch (Exception ex) →` captura la excepción.

Línea 55: `Log.Fatal(ex, "La aplicación ha terminado debido a una excepción no controlada"); →` registra el error fatal.

Línea 57: `finally →` bloque final.

Línea 59: `Log.CloseAndFlush(); →` cierra el logger y vacía los buffers.

Error común: si se olvida `Log.CloseAndFlush()`, los mensajes pendientes pueden perderse al cerrar la aplicación.

■ Paso 4: Crear el caso de uso de logging y diagnóstico

Crear el archivo `src/AceriaData.Application/UseCases/LoggingDiagnosticoUseCase.cs`:

```
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using Microsoft.Extensions.Logging;

namespace AceriaData.Application.UseCases;

public class LoggingDiagnosticoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    private readonly ILogger<LoggingDiagnosticoUseCase> _logger;

    public LoggingDiagnosticoUseCase(IUnidadDeTrabajo unidad, ILogger<LoggingDiagnosticoUseCase> logger)
    {
        _unidad = unidad;
        _logger = logger;
    }

    public void Ejecutar()
    {
        _logger.LogInformation("=== LOGGING Y DIAGNÓSTICO ===");

        DemostrarLoggingDeConsultas();
        DemostrarLoggingDeEscritura();
    }

    private void DemostrarLoggingDeConsultas()
    {
        _logger.LogInformation("--- Consultas ---");

        var ordenes = _unidad.Ordenes.ObtenerTodas();
        _logger.LogInformation("Se han cargado {Total} órdenes", ordenes.Count);

        var pendientes = _unidad.Ordenes.ObtenerPendientes();
        _logger.LogInformation("Se han cargado {Total} órdenes pendientes", pendientes.Count);
    }

    private void DemostrarLoggingDeEscritura()
    {
        _logger.LogInformation("--- Escritura ---");
    }
}
```

```

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-LOG-001",
            Cliente = "Constructora del Norte",
            Estado = "Pendiente",
            FechaCreacion = DateTime.Now
        };

        _unidad.Ordenes.Agregar(orden);
        var filas = _unidad.Guardar();

        _logger.LogInformation("Se han insertado {Filas} filas. Orden creada: {NumeroOrden}", filas,
            orden.NumeroOrden);
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 3: using Microsoft.Extensions.Logging; → importa el logging.

Línea 5: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 7: public class LoggingDiagnosticoUseCase → declara el caso de uso.

Línea 9: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: private readonly ILogger _logger; → campo del logger.

Línea 12: public LoggingDiagnosticoUseCase(IUnidadDeTrabajo unidad, ILogger logger) → constructor.

Línea 14: _unidad = unidad; → asigna la unidad de trabajo.

Línea 15: _logger = logger; → asigna el logger.

Línea 18: public void Ejecutar() → declara el método principal.

Línea 20: _logger.LogInformation("=== LOGGING Y DIAGNÓSTICO ==="); → registra la cabecera.

Línea 22: DemostrarLoggingDeConsultas(); → llama al método de consultas.

Línea 23: DemostrarLoggingDeEscritura(); → llama al método de escritura.

Línea 26: private void DemostrarLoggingDeConsultas() → declara el método.

Línea 28: _logger.LogInformation("--- Consultas ---"); → registra la cabecera.

Línea 30: var ordenes = _unidad.Ordenes.ObtenerTodas(); → carga las órdenes.

Línea 31: _logger.LogInformation("Se han cargado {Total} órdenes", ordenes.Count); → registra el número.

Línea 33: var pendientes = _unidad.Ordenes.ObtenerPendientes(); → carga las pendientes.

Línea 34: _logger.LogInformation("Se han cargado {Total} órdenes pendientes", pendientes.Count); → registra el número.

Línea 37: private void DemostrarLoggingDeEscritura() → declara el método.

Línea 39: _logger.LogInformation("--- Escritura ---"); → registra la cabecera.

Línea 41: var orden = new OrdenFabricacion → crea la orden.

Línea 49: _unidad.Ordenes.Agregar(orden); → agrega la orden.

Línea 50: var filas = _unidad.Guardar(); → guarda los cambios.

Línea 52: _logger.LogInformation("Se han insertado {Filas} filas. Orden creada: {NumeroOrden}", filas, orden.NumeroOrden); → registra el resultado con propiedades estructuradas.

Error común: si el logger no se inyecta, el caso de uso no compila. Se debe inyectar ILogger en el constructor.

Paso 5: Configurar el filtrado de Serilog

Modificar la configuración de Serilog en el método Main:

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Database.Command",
Serilog.Events.LogEventLevel.Information)
    .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Update",
Serilog.Events.LogEventLevel.Information)
    .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Query", Serilog.Events.LogEventLevel.Warning)
    .WriteTo.Console()
    .WriteTo.File("logs/aceria-.log", rollingInterval: RollingInterval.Day)
    .CreateLogger();
```

Línea 1: Log.Logger = new LoggerConfiguration() → crea la configuración.

Línea 2: .MinimumLevel.Information() → nivel mínimo global.

Línea 3: .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Database.Command", Serilog.Events.LogEventLevel.Information) → nivel para comandos.

Línea 4: .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Update", Serilog.Events.LogEventLevel.Information) → nivel para actualizaciones.

Línea 5: .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Query", Serilog.Events.LogEventLevel.Warning) → nivel para consultas.

Línea 6: .WriteTo.Console() → escribe en la consola.

Línea 7: .WriteTo.File("logs/aceria-.log", rollingInterval: RollingInterval.Day) → escribe en un archivo diario.

Línea 8: .CreateLogger(); → crea el logger.

Error común: si se filtran demasiado los mensajes, se pueden perder los mensajes importantes. Se debe ajustar el nivel según el entorno.

Paso 6: Registrar el caso de uso en el contenedor

Modificar el método Main para registrar el caso de uso:

```
services.AddScoped<LoggingDiagnosticsUseCase>();
```

Línea 1: services.AddScoped(); → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

Paso 7: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<LoggingDiagnosticsUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var useCase = scope.ServiceProvider.GetRequiredService(); → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 8: Insertar datos de prueba iniciales

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 3, 10) }
    };

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el DbContext.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base de datos.

Línea 5: `context.Database.Migrate();` → aplica las migraciones.

Línea 7: `var ordenes = new List` → crea la lista.

Línea 9: `new OrdenFabricacion { ... }` → primera orden.

Línea 10: `new OrdenFabricacion { ... }` → segunda orden.

Línea 11: `new OrdenFabricacion { ... }` → tercera orden.

Línea 12: `};` → cierra la lista.

Línea 14: `context.OrdenesFabricacion.AddRange(ordenes);` → registra las órdenes.

Línea 15: `context.SaveChanges();` → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

| Paso 9: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen los mensajes de logging en la consola y en el archivo `logs/aceria-YYYYMMDD.log`. Se observan las sentencias SQL, los mensajes del caso de uso y las operaciones de escritura.

| Paso 10: Analizar la salida

La salida del programa muestra información como la siguiente:

```
[12:00:00 INF] Iniciando la aplicación AceriaData
[12:00:01 INF] === LOGGING Y DIAGNÓSTICO ===
[12:00:01 INF] --- Consultas ---
[12:00:01 INF] Executed DbCommand (2ms) [Parameters=[], CommandType='Text', CommandTimeout='60']
SELECT [o].[Id], [o].[NumeroOrden], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
[12:00:01 INF] Se han cargado 3 órdenes
[12:00:01 INF] Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='60']
SELECT [o].[Id], [o].[NumeroOrden], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente' AND [o].[IsDeleted] = CAST(0 AS bit)
[12:00:01 INF] Se han cargado 2 órdenes pendientes
[12:00:01 INF] --- Escritura ---
[12:00:02 INF] Executed DbCommand (5ms) [Parameters=[@p0='OF-LOG-001', ...], CommandType='Text',
CommandTimeout='60']
INSERT INTO [OrdenesFabricacion] ([NumeroOrden], [Cliente], [Estado], [FechaCreacion], [IsDeleted])
VALUES (@p0, @p1, @p2, @p3, @p4);
[12:00:02 INF] Se han insertado 1 filas. Orden creada: OF-LOG-001
[12:00:02 INF] La aplicación ha terminado
```

La primera sección muestra el inicio de la aplicación. La segunda sección muestra las consultas con las sentencias SQL. La tercera sección muestra la escritura con el INSERT. La cuarta sección muestra el cierre.

Observaciones: el logging registra las sentencias SQL, los parámetros y los mensajes del caso de uso. Serilog escribe en la consola y en el archivo. Los mensajes estructurados incluyen propiedades tipadas.

Paso 11: Diagnosticar un error común

Modificar la configuración de Serilog para no filtrar los mensajes de EF Core:

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .WriteTo.Console()
    .WriteTo.File("logs/aceria-.log", rollingInterval: RollingInterval.Day)
    .CreateLogger();
```

Resultado esperado: se registran todos los mensajes de EF Core, incluyendo los de nivel Debug y Trace. El log se vuelve muy verbose y difícil de analizar.

Solución: filtrar los mensajes por categoría con `MinimumLevel.Override`.

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Database.Command",
Serilog.Events.LogEventLevel.Information)
    .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Query", Serilog.Events.LogEventLevel.Warning)
    .WriteTo.Console()
    .WriteTo.File("logs/aceria-.log", rollingInterval: RollingInterval.Day)
    .CreateLogger();
```

Resultado esperado con la solución: solo se registran los mensajes de las categorías configuradas con los niveles adecuados.

Errores comunes del ejercicio

Error Causa Solución Serilog no configurado Falta el paquete o la configuración Instalar los paquetes y configurar Logger no inyectado Falta el ILogger en el constructor Inyectar el logger Mensajes no registrados Nivel mínimo demasiado alto Ajustar el nivel Log demasiado verbose No se filtran las categorías Usar `MinimumLevel.Override` Archivo de log no creado Falta el directorio logs Crear el directorio o configurar la ruta Log.CloseAndFlush no llamado Los mensajes pendientes se pierden Llamar en el finally

Retos resueltos: Logging estructurado con propiedades tipadas

Reto: Añadir logging estructurado al repositorio para registrar cada operación de escritura con propiedades tipadas. Usar ILogger en el repositorio y registrar el número de orden, el cliente y el número de filas afectadas.

Solución paso a paso:

Paso 1: Modificar el repositorio para recibir ILogger:

```
using Microsoft.Extensions.Logging;

public class OrdenRepositorio : Repositorio<OrdenFabricacion>, IOrdenRepositorio
{
    private readonly ILogger<OrdenRepositorio> _logger;

    public OrdenRepositorio(AceriaDbContext context, ILogger<OrdenRepositorio> logger) : base(context)
    {
        _logger = logger;
    }

    public override void Agregar(OrdenFabricacion orden)
    {
        _logger.LogInformation("Agregando orden {NumeroOrden} del cliente {Cliente}",
            orden.NumeroOrden, orden.Cliente);
        base.Agregar(orden);
    }
}
```

Línea 1: using Microsoft.Extensions.Logging; → importa el logging.

Línea 3: public class OrdenRepositorio : Repositorio, IOrdenRepositorio → declara el repositorio.

Línea 5: private readonly ILogger _logger; → campo del logger.

Línea 7: public OrdenRepositorio(AceriaDbContext context, ILogger logger) : base(context) → constructor.

Línea 9: _logger = logger; → asigna el logger.

Línea 12: public override void Agregar(OrdenFabricacion orden) → sobrescribe el método.

Línea 14: _logger.LogInformation("Agregando orden {NumeroOrden} del cliente {Cliente}", orden.NumeroOrden, orden.Cliente); → registra el mensaje estructurado.

Línea 15: base.Agregar(orden); → llama al método base.

Paso 2: Registrar el repositorio en el contenedor:

```
services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
```

Línea 1: services.AddScoped<IOrdenRepositorio, OrdenRepositorio>(); → registra el repositorio.

Paso 3: Ejecutar el caso de uso y verificar que los mensajes aparecen con las propiedades tipadas.

Resultado esperado: los mensajes de logging incluyen las propiedades NumeroOrden y Cliente, que se pueden consultar y filtrar en el sistema de logging.

Ampliación técnica obligatoria — diagnóstico completo, archivo, EventCounters y telemetría

La fuente original cubre ILogger, Serilog y logging estructurado, pero el contrato del curso exige completar la observabilidad de EF Core 8 con evidencia ejecutable adicional.

Paso 12: Configurar un sink de archivo con rotación y retención

Añade el sink de archivo y configura límites explícitos:

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .WriteTo.Console()
    .WriteTo.File(
        path: "logs/aceriadata-.log",
        rollingInterval: RollingInterval.Day,
        fileSizeLimitBytes: 10 * 1024 * 1024,
        rollOnFileSizeLimit: true,
        retainedFileCountLimit: 7,
        shared: true)
    .CreateLogger();
```

Genera suficientes mensajes para comprobar que se crea el archivo y que la configuración de rotación está activa. No basta con declarar las opciones: el laboratorio debe verificar al menos la existencia del archivo y registrar la configuración usada.

Paso 13: Implementar un observador completo de DiagnosticListener

`DiagnosticListener` permite observar eventos EF Core del proceso. No sustituye a `ILogger`; es un mecanismo de diagnóstico distinto.

```
using System.Diagnostics;

public sealed class EfDiagnosticObserver :
    IObservable<DiagnosticListener>,
    IObservable<KeyValuePair<string, object?>>, IDisposable
{
    private readonly List<IDisposable> _subscriptions = new();
    private readonly Action<string> _write;

    public EfDiagnosticObserver(Action<string> write) => _write = write;

    public void Start() =>
        _subscriptions.Add(DiagnosticListener.AllListeners.Subscribe(this));

    public void OnNext(DiagnosticListener listener)
    {
        if (listener.Name == "Microsoft.EntityFrameworkCore")
            _subscriptions.Add(listener.Subscribe(this));
    }

    public void OnNext(KeyValuePair<string, object?> evt)
    {
        if (evt.Key.Contains("CommandExecuting", StringComparison.Ordinal) ||
            evt.Key.Contains("CommandExecuted", StringComparison.Ordinal) ||
            evt.Key.Contains("SaveChanges", StringComparison.Ordinal))
        {
            _write($"EF-DIAG | {evt.Key}");
        }
    }

    public void OnError(Exception error) => _write($"EF-DIAG-ERROR | {error.Message}");
    public void OnCompleted() { }

    public void Dispose()
    {
        foreach (var subscription in _subscriptions)
            subscription.Dispose();
        _subscriptions.Clear();
    }
}
```

El escenario debe iniciar el observador, ejecutar una consulta y un `SaveChanges`, comprobar que se recibieron eventos y liberar todas las suscripciones.

Paso 14: Observar los EventCounters de EF Core 8

En EF Core 8 la práctica usa **EventCounters**. Las métricas basadas en `System.Diagnostics.Metrics` con nombres `microsoft.entityframeworkcore.*` se introdujeron posteriormente y no deben presentarse como una característica de EF Core 8.

Muestra el PID del proceso:

```
Console.WriteLine($"PID: {Environment.ProcessId}");
```

Desde otra terminal:

```
dotnet counters monitor --counters Microsoft.EntityFrameworkCore -p <PID>
```

Durante la ejecución deben poder observarse contadores como `Active DbContexts`, `Queries`, `SaveChanges`, `Query Cache Hit Rate` y `Optimistic Concurrency Failures` cuando el escenario los produzca.

Paso 15: Añadir Application Insights como destino mediante Azure Monitor/OpenTelemetry

Para una aplicación de consola no se debe asumir que `AddApplicationInsightsTelemetry()` configura mágicamente un host ASP.NET. Usa un pipeline `OpenTelemetry` compatible con .NET y el exportador de Azure Monitor cuando exista una cadena de conexión.

```
dotnet add package OpenTelemetry
dotnet add package Azure.Monitor.OpenTelemetry.Exporter
```

```
using Azure.Monitor.OpenTelemetry.Exporter;
using OpenTelemetry;
using OpenTelemetry.Trace;

var aiConnectionString = Environment.GetEnvironmentVariable(
    "APPLICATIONINSIGHTS_CONNECTION_STRING");

TracerProvider? azureMonitor = null;
if (!string.IsNullOrEmpty(aiConnectionString))
{
    azureMonitor = Sdk.CreateTracerProviderBuilder()
        .AddSource("AceriaData")
        .AddAzureMonitorTraceExporter(options =>
            options.ConnectionString = aiConnectionString)
        .Build();
}
```

La demo de Azure Monitor es opcional si no existe una cadena de conexión real. El laboratorio local sigue siendo verificable mediante `ILogger`, `Serilog`, `DiagnosticListener`, `EventCounters` y archivo. No se afirma que los eventos específicos de `DiagnosticListener` de EF Core se exporten automáticamente como trazas de Application Insights; el pipeline de telemetría debe instrumentarse de forma explícita.

Paso 16: Evidencia final de 5.10

La ejecución debe dejar evidencia observable de:

1. logs de EF Core filtrados por categoría/nivel;
2. logging estructurado con propiedades;
3. archivo de log y configuración de rotación/retención;
4. eventos recibidos mediante `DiagnosticListener`;
5. `EventCounters` de EF Core 8 observables con `dotnet-counters`;

6. configuración opcional de exportación a Azure Monitor/Application Insights mediante OpenTelemetry.

| Analogía final

El logging y el diagnóstico en una acería son como el sistema de registro del panel de control. Cada operación que se realiza en la planta se anota en el registro: qué se hizo, cuándo, con qué parámetros y cuánto tardó. El registro permite detectar problemas, analizar el rendimiento y auditar los cambios. El logging estructurado es como el registro con columnas: cada campo tiene su propio espacio y se puede consultar por separado. Serilog es como el sistema de registro que escribe en varios destinos: la consola, el archivo y el servicio de monitorización. El filtrado por categoría es como elegir qué secciones del registro se quieren consultar. El diagnóstico es como el análisis del registro: se buscan patrones, se detectan anomalías y se toman decisiones. Así funcionan el logging y el diagnóstico en EF Core: se registra, se filtra y se analiza para monitorizar el comportamiento de la aplicación.

| Resultado esperado

Al final del ejercicio, deberías haber:

Instalado los paquetes de Serilog.

Configurado Serilog en el método Main.

Configurado el filtrado por categoría con `MinimumLevel.Override`.

Creado el caso de uso `LoggingDiagnosticoUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba iniciales.

Ejecutado el proyecto y verificado los logs en la consola y en el archivo.

Diagnosticado el error de no filtrar las categorías.

Añadido logging estructurado al repositorio.

| Conexión con el siguiente punto

En este punto se han estudiado el logging y el diagnóstico en EF Core, incluyendo la integración con ILogger, la configuración de Serilog, el filtrado por categoría y el logging estructurado. Se ha comprobado que el logging permite monitorizar el comportamiento de EF Core en producción. En el siguiente punto se estudiará el testing con EF Core: InMemory y SQLite in-memory.
